

CSE222 / BIL505 HOMEWORK #7-8

Sorting Algorithms, Adjacency Matrix, and Graph Coloring

GEBZE TECHNICAL UNIVERSITY COMPUTER ENGINEERING DEPARTMENT

Project Report

Submitted by: Fatih Emre OĞAN

Student ID: 230104004090

Submitted to: Enes Abdülhalik

Submission Date: May 25, 2025

Introduction

In this project, I completed CSE222/BIL505 Homework 7-8, which focused on implementing sorting algorithms, a graph class with an adjacency matrix structure, and integrating them with a greedy graph coloring algorithm. I implemented three sorting algorithms: GTUInsertSort (using insertion sort), GTUQuickSort (using quicksort with random pivot selection), and as a bonus, GTUSelectSort (using selection sort, +10 points). For the graph implementation, I developed the MatrixGraph class, adhering to the GTUGraph interface, and used an adjacency matrix structure to store edges. This class utilized the AdjacencyVect class, which I implemented based on the Collection interface. As a bonus, I fully implemented all methods of AdjacencyVect, including removeAll, retainAll, and toArray, earning an additional +10 points. I integrated my implementations with the provided greedy graph coloring algorithm, which colors vertices based on their degrees in descending order. To ensure the correctness of my code, I wrote automated unit tests for all classes, including GTUInsertSort, GTUQuickSort, GTUSelectSort, MatrixGraph, AdjacencyVect, and the graph coloring components (GreedyGCA, GCASolution, GreedyVertex), fulfilling the bonus requirement for automated unit testing (+20 points). The program reads a graph from an input file and generates three output files: sorted sums of edge pairs (in descending order), adjacency information of the graph, and the graph coloring solution. Through this project, I gained practical experience in sorting algorithms, graph data structures, and graph coloring techniques.

Environment

I developed and edited my code using Visual Studio Code (VS Code) as my primary integrated development environment (IDE) on a Windows system. For compilation and testing, I used Git Bash to execute the provided makefile commands, including make clean, make collect, make build, and make run. To ensure compatibility with the assignment's requirements, I installed OpenJDK 11 on my system and verified its version to match the specified environment (Ubuntu 22.04 and OpenJDK 11). I tested my code locally using Git Bash, ensuring that the makefile commands worked seamlessly and that my setup was compatible with the expected evaluation environment

Complexity Analysis

In this project, I analyzed the time complexity of key methods across different classes. For the sorting algorithms, GTUInsertSort has a time complexity of $O(n^2)$ due to nested loops in the insertion sort algorithm, where n is the array size. GTUQuickSort achieves an average time complexity of $O(n \log n)$ with a random pivot, but can degrade to $O(n^2)$ in the worst case due to poor pivot selection. GTUSelectSort, implemented as a bonus, also has a time complexity of $O(n^2)$ because selection sort repeatedly finds the minimum element. In Main.java, the writeSortedSums method has a complexity of $O(E \log E)$, where E is the number of edges, due to sorting the edge sums using the provided sorter. For MatrixGraph, operations like setEdge and getEdge are $O(1)$ as they directly access the adjacency matrix, while getNeighbors is $O(V)$, where V is the number of vertices, due to iterating over the adjacency vector to find neighbors. In AdjacencyVect, methods like add, remove, and contains are $O(1)$ since they access a fixed-size boolean array, but the iterator takes $O(V)$ to iterate over true elements. The GreedyGCA.solve method has a complexity of $O(V \log V + V^2)$, where V is the number of vertices, due to sorting vertices by degree ($O(V \log V)$) and checking neighbors for coloring (up to $O(V^2)$). Finally, GCASolution.writeSolution has a complexity of $O(V \log V)$ due to sorting vertices within each color group.

Testing

I tested my software by developing automated unit tests for all implemented classes, fulfilling the bonus requirement for automated unit testing (+20 points). These tests covered sorting algorithms (GTUInsertSort, GTUQuickSort, GTUSelectSort), graph implementations (MatrixGraph, AdjacencyVect), and graph coloring components (GreedyGCA, GCASolution, GreedyVertex). For instance, GTUInsertSortTest verified that the array [1, 2, 3, 4] was correctly sorted, while GreedyGCATest confirmed that adjacent vertices (e.g., 0 and 1) were assigned different colors (e.g., 1 and 0). I also tested the main program by providing an input file in the specified format and verified that it generated the required output files: sorted sums (in descending order), adjacency information, and coloring solutions, all matching the expected format. All tests passed successfully, as indicated by the output "Test PASSED" for each case. Additionally, you can find the test scenarios executed using the make test command below. For further evidence, I have included screenshots of the test outputs.

```
fatih@fatih MINGW64 ~/Desktop/CSE222/CSE222_Homeworks/hw7-8
$ make test
find src -name "*.java" > sources.txt
mkdir -p build doc
javac -d build -sourcepath src @sources.txt
Running unit tests...
Running DSA.Sorting.GTUSelectSortTest...
Sorted array: 1 2 5 8
Expected: 1 2 5 8
Test PASSED

Running DSA.Sorting.GTUInsertSortTest...
Sorted array: 1 2 3 4
Expected: 1 2 3 4
Test PASSED

Running DSA.Sorting.GTUQuickSortTest...
Sorted array: 1 5 7 8 10
Expected: 1 5 7 8 10
Test PASSED

Running DSA.Graphs.MatrixGraphTest...
Edge (0,1): true
Edge (1,0): true
Neighbors of 0: 1
Test PASSED

Running DSA.Graphs.AdjacencyVectTest...
Contains 1 after add: true
Contains 1 after remove: false
Size: 0
Test PASSED

Running DSA.Graphs.GCA.GreedyGCATest...
Color of 0: 1
Color of 1: 0
No conflict: true
Test PASSED
```

```
Running DSA.Graphs.GCA.GCASolutionTest...
Vertices in color 1: DSA.Graphs.AdjacencyVect@1f32e575
Vertices in color 2: DSA.Graphs.AdjacencyVect@1be6f5c3
Test PASSED

Running Main.TestScenarioTest...
File name: test.txt
Sorter: true
Test PASSED

Running DSA.Graphs.GCA.GreedyVertexTest...
Compare result: -1
Test PASSED
```

AI Usage

I used Grok, to assist me in several aspects of this project. Specifically, I relied on Grok while creating the makefile, as it helped me understand and correctly set up the build and run commands required for the assignment. In the sorting algorithms section, I sought Grok's assistance to better understand the implementation of GTUInsertSort, GTUQuickSort, and GTUSelectSort, ensuring their correctness and efficiency. Additionally, during the graph coloring part, Grok provided valuable guidance in integrating my sorting and graph implementations with the provided GreedyGCA algorithm, helping me debug issues and verify the coloring solution format. This support ensured that my work aligned with the assignment requirements.

Challenges

I faced several challenges during this project. Initially, while implementing the AdjacencyVect class, I encountered type conversion errors, especially when processing the Collection interface methods due to incorrect data type usage. Another challenge was ensuring the correct format for output files such as color.txt, where I had difficulty separating the vertex and color counts into separate rows. While working on the sorting algorithms, I encountered minor bugs and issues such as incorrect or reversed sorting orders. To solve this, I entered various values and compared my outputs with my colleagues; I noticed inconsistencies that led me to reconsider and improve the implementations of GTUInsertSort, GTUQuickSort, and GTUSelectSort. I also faced issues with library restrictions as the assignment allowed only certain libraries; this required me to do manual conversions in several places to avoid using prohibited libraries, which was difficult. Integrating the sorting algorithms with the GreedyGCA algorithm also required debugging to align the vertex sorting with the degree-based coloring process. I tried to overcome these issues by referring to Java documentation for type handling and library usage and to identify and fix errors effectively.

Conclusion

Through this project, I gained a deeper understanding of sorting algorithms, including insertion sort, quicksort, and selection sort, and their practical implementation in the GTUInsertSort, GTUQuickSort, and GTUSelectSort classes. I learned how to use adjacency matrices to represent graphs and effectively implemented the MatrixGraph class using the AdjacencyVect class. The greedy graph coloring algorithm taught me optimization techniques and the importance of vertex ordering based on degrees. Writing unit tests for all classes improved my ability to verify code correctness, enhancing my skills in software testing. This experience, which I completed independently, solidified my knowledge of data structures, algorithms, and their practical applications.